DISCRETE MATHEMATICS

1. This question carries 30% of the marks.

   For each of the following statements, state whether it is true or false and provide a supporting proof.

   a) $3n-5=\Theta(n)$. [3]

   Answer:
   True. $\lim_{n\to\infty}\frac{3n-5}{n}=\lim_{n\to\infty}\left(3-\frac{5}{n}\right)=3$, so the ratio to n tends to a positive finite constant and $3n-5=\Theta(n)$.

   b) $n^2+7n+3=O(n)$. [3]

   Answer:
   False. $\lim_{n\to\infty}\frac{n^2+7n+3}{n}=\lim_{n\to\infty}\left(n+7+\frac{3}{n}\right)=\infty$, so the ratio to n cannot be bounded by a finite constant and the function is not $O(n)$.

   c) $2^n=\Omega(n^3)$. [3]

   Answer:
   True. Using the results from lectures, we know that any polynomial is $o(2^n)$ so $\lim_{n\to\infty}\frac{n^3}{2^n}=0$, so $\lim_{n\to\infty}\frac{2^n}{n^3}=\infty$, so $2^n=\omega(n^3)$ and therefore also $\Omega(n^3)$.

   d) $n^2+n=\Theta(n\log n)$. [3]

   Answer:
   False. $\lim_{n\to\infty}\frac{n^2+n}{n\log n}=\lim_{n\to\infty}\frac{n+1}{\log n}=\infty$, so n^2+n grows faster than $n\log n$ and is not $\Theta(n\log n)$.

   e) $\log_2 n=O(\log_3 n)$. [3]

   Answer:
   True. By change of base formula, $\log_2 n=\frac{\log_3 n}{\log_3 2}$, so $\lim_{n\to\infty}\frac{\log_2 n}{\log_3 n}=\frac{1}{\log_3 2}$, which is a positive constant. Thus $\log_2 n=\Theta(\log_3 n)$ which implies $\log_2 n=O(\log_3 n)$.

2. This question carries 20% of the marks.

   Consider the following pseudocode for computing the maximum value of an array $A[1\ldots n]$ of integers.

```
function MAX(A)
    n ← length(A)
    m ← A[1]
    for i ← 2 to n do
        if A[i] > m then
            m ← A[i]
        end if
    end for
    return m
end function
```

   a) State a loop invariant for the `for` loop.

Answers

Recall: a loop invariant is a property that holds at the start of each iteration of the loop and is preserved through each iteration, and that is useful for proving the correctness of the algorithm. [3]

Answer:

Loop invariant: At the start of each iteration with index i (and after any updates), m equals the maximum value among $A[1 \ldots i-1]$.

b) Prove initialization and maintenance of your invariant. [5]

Answer:

Initialization: Before the loop, $i = 2$ and $m = A[1]$, so m is the maximum of $A[1 \ldots 1]$. Maintenance: Assume the invariant holds at the start of iteration i. If $A[i] > m$, setting $m \leftarrow A[i]$ makes m the maximum of $A[1 \ldots i]$. If not, m remains the maximum of $A[1 \ldots i-1]$, which is also the maximum of $A[1 \ldots i]$. Thus the invariant holds for the next iteration.

c) Use the invariant to prove the algorithm returns the maximum value in $A[1 \ldots n]$. [2]

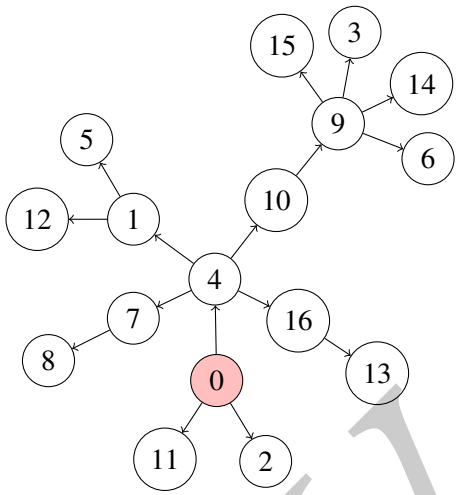
Answer:

Termination: When the loop ends, $i = n+1$, so by the invariant m is the maximum of $A[1 \ldots n]$, which is the claimed result.

Answers

3. This question carries 50% of the marks.

Consider the following problem for scheduling a list of n tasks to run on m processors. Each task takes a unit of time to run. Each processor can run at most one task at a time. Some tasks cannot start until other tasks have finished. We define a graph $G = (V, E)$ with nodes V corresponding to tasks, and a directed edge $(i, j) \in E$ from node i to node j if task i depends on task j (i.e., j must finish before i can start). The problem is to find a schedule for all tasks that respects the dependencies and minimizes the total time to complete all tasks. An example of such a graph is shown below, where task 0 (in pink) is the final task to be completed, which depends on tasks 2, 4 and 11, and so on.



Hu's algorithm gives an optimal solution for this problem when the graph is a tree with a single root node, as in the example above (0 is the root node). First, we define the *level* of each node to be the shortest distance from the root node. In the example above the root node 0 has level 0, nodes 2, 4 and 11 have level 1, and so on. Hu's algorithm says that whenever a processor becomes available, we should schedule any task that satisfies the conditions (C_1) it has not yet been started, (C_2) all its dependencies have completed, and (C_3) it has the highest level among all tasks satisfying C_1 and C_2.

a) Apply a breadth-first search to compute the level of each node in the example graph above.

i) List all the nodes at each level. Give your answer in the form of a table with one row for each level, and list all the nodes in that level in increasing order. [5]

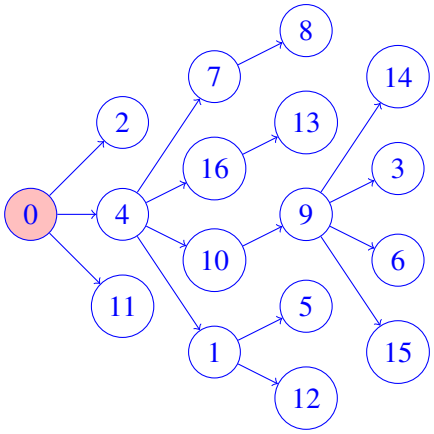
Answer:
Level 0: [0]
Level 1: [2, 4, 11]
Level 2: [1, 7, 10, 16]
Level 3: [5, 8, 9, 12, 13]
Level 4: [3, 6, 14, 15]

To get this result, it may be helpful to draw the tree with the nodes arranged in levels, as shown below.

ii) In the case of an infinite number of processors ($m = \infty$), what is the minimum time to complete all tasks and why? [2]

Answer:
The minimum time is 5 because in the first time step you could complete all the level 4 tasks which have no dependencies, then in the next time step all the level 3 tasks, and so on.

Answers



Note (not required for full marks) that with infinite processors you can pick off all the tasks with no dependencies at all levels, but this doesn't reduce the time because there must be at least one path from the start node to one of the nodes of level 4, and this path must have length 5, so it must take at least 5 time steps to complete all tasks.

b) Apply Hu's algorithm to the example graph in the case $m = 3$. Use the extra condition (C_4) that we pick the task with the lowest index among all tasks satisfying $C_1 - C_3$. List the tasks scheduled at each time step for each processor in a table, and give the total time to complete all tasks. [3]

Answer:

Time Step	Processor 1	Processor 2	Processor 3
1	3	6	14
2	15	5	8
3	9	12	13
4	1	7	10
5	16	2	4
6	11		
7	0		

Total time: 7

c) We want to prove that Hu's algorithm always gives an optimal schedule. The first step is to compute a lower bound on the time to complete all tasks that any schedule must satisfy. To find this lower bound, let V_L be the set of nodes whose level is greater than or equal to L. Let h be the maximum level of any node.

i) If we could ignore all dependencies between tasks in V_L and just schedule them as if they were independent, but using only m processors, how many time steps would it take to complete all tasks in V_L? [2]

Answer:

It would take $\lceil |V_L|/m \rceil$ time steps. If $|V_L|$ is divisible by m then we can fully utilise all m processors at each time step. If not, we will waste some time for the final time step, so we need to round up to the nearest integer.

ii) For the remaining tasks in $V - V_L$, how many time steps would it take to complete all those tasks if we had an infinite number of processors? [2]

Answer:

It would take L time steps because we could complete all level $L-1$

Answers

tasks at the first time step, then all level $L-2$ tasks at the second time step, and so on until we complete all level 0 tasks at the L-th time step.

iii) Using your answers to the previous two parts, and considering all possible values of L, give a lower bound on the time to complete all tasks that any schedule must satisfy. [1]

Answer:

We get a lower bound of $\lceil |V_L|/m \rceil + L$ for each value of L because the first term gives a lower bound on the time to complete all tasks in V_L and the second term gives a lower bound on the time to complete all tasks in $V - V_L$. Since this must hold for all values of L, we get a lower bound of

$$\max_{0 \le L \le h} \lceil |V_L|/m \rceil + L.$$

iv) Compute this lower bound for the example graph above, and show that it matches the time taken by Hu's algorithm. [1]

Answer:

V_0 has 17 nodes so $\lceil |V_0|/3 \rceil + 0 = 6$.
V_1 has 16 nodes so $\lceil |V_1|/3 \rceil + 1 = 7$.
V_2 has 13 nodes so $\lceil |V_2|/3 \rceil + 2 = 7$.
V_3 has 9 nodes so $\lceil |V_3|/3 \rceil + 3 = 6$.
V_4 has 4 nodes so $\lceil |V_4|/3 \rceil + 4 = 6$.
So the maximum is 7, which matches the time taken by Hu's algorithm.

d) We will complete this proof by showing this lower bound is attained.

i) Let S_t be the set of tasks scheduled by Hu's algorithm at time step t. Let A be the set of tasks that can be scheduled at a given moment (i.e. they satisfy C_1 and C_2). Show that $|S_0| \ge |S_1| \ge |S_2| \ge \ldots$.

Hint: consider what can happen to $|A|$ each time a scheduling operation takes place in Hu's algorithm. [2]

Answer:

Throughout Hu's algorithm, at each scheduling operation the size of $|A|$ can only stay the same or decrease by 1. If completing a task makes a new task available (it was the last dependency for that task) then $|A|$ stays the same, otherwise $|A|$ decreases by 1 because 1 task was removed and no new task was added. It's not possible for more than one task to be added because we're assuming the graph is a tree. Hu's algorithm will always schedule a task if one is available, so $|S_t| = \min(m, |A|)$ at each time step, and will therefore always stay the same or decrease at each time step.

ii) Show that from one time step to the next, the highest level among all remaining tasks cannot decrease by more than 1. [2]

Answer:

Pick a task v of the highest level – say ℓ – that was scheduled to be completed in this step. Since this task has level ℓ there must be a path from the root node to v of length ℓ. The preceding node v' in this path has level $\ell - 1$. This node v' cannot be completed in the same step as v because it depends on v having been completed so it would not have been scheduled. Therefore, at the end of this step there will be at least one remaining task of level $\ell - 1$ and therefore the highest level among all remaining tasks must be at least $\ell - 1$.

Answers

iii) Let k be the last time step at which Hu's algorithm schedules precisely m tasks all from the same level, and let L be that level. Let L' be the highest level among all tasks remaining after time step k. Show that the schedule takes precisely $L'+1$ steps after stage k. [2]

Answer:

We know from the earlier part that the highest level cannot decrease by more than 1 at each time step, so it must take at least $L'+1$ steps to complete all remaining tasks (note that we have to include the time taken for the final step which is why it is $L'+1$ and not L'). If the highest level decreases by 0 over a time step, it means that all tasks in that time step were of the same level, which would contradict our choice of k. Therefore, the highest level remaining must decrease by precisely 1 at each time step, so it must take precisely $L'+1$ steps to complete all remaining tasks.

iv) Show that for all stages up to and including stage k, all processors are fully utilised. [1]

Answer:

We know that the set of tasks A that could be scheduled at any scheduling operation always stays the same or decreases by 1. We know that at step k the size of this set must have been at least m because we scheduled m tasks. Therefore, at all previous steps the size of this set must also have been at least m, so all processors must have been fully utilised at all previous steps.

v) Show that the schedule takes precisely $\lceil |V_L|/m \rceil + L$ steps, demonstrating that Hu's algorithm achieves the lower bound.

Hint: consider the two cases where S_k uses up all tasks at level L, and where it does not. [2]

Answer:

In the first case, we have that $L' = L-1$ because all tasks of level L were used up in step k. All processors are fully utilised in stages up to and including k and all tasks are from V_L and therefore completing the tasks in V_L must have taken $\lceil |V_L|/m \rceil$ steps. After stage k we already showed that it takes $L' = L-1$ steps to complete all remaining tasks, so the total time is $\lceil |V_L|/m \rceil + L$ as claimed.

In the second case, $L = L'$ because there are some tasks of level L that were not scheduled in step k. The time taken must then be $L'+1+(|S_0|+\ldots+|S_k|)/m$. We know that there are fewer than m tasks of level L scheduled after step k (otherwise k would have been higher) and therefore this time taken is just $\lceil |V_L|/m \rceil + L$ as claimed.

Answers